
Proximal Splitting Methods for Hybrid Differentiable Models

Abdel-Rahim Mezidi¹

Jordan Patracone¹

Amaury Habrard^{1 2}

Abstract

Operator splitting methods are at the foundation of many numerical solvers for partial differential equations. In parallel, unrolled and hybrid learning-based architectures have been introduced to enhance classical solvers, but their design is rarely linked to the underlying problem structure. In this work, we propose a unifying framework that explicitly links operator splitting algorithms from optimization with unrolled hybrid architectures. We show that each operator splitting scheme naturally defines an unrolled architecture, which recovers a wide range of existing plug-and-play and hybrid models as special cases. Using this framework, we design new unrolled hybrid architectures and validate them on benchmark fluid dynamics simulations, where they achieve improved accuracy and stability.

1. Introduction

Hybrid differentiable models that combine neural networks with physics-based numerical solvers have shown promise in scientific machine learning by balancing data-driven flexibility with physical consistency (Yin et al., 2021; Tompson et al., 2017; Um et al., 2020). Existing approaches include *embedded* methods that replace solver components (Tompson et al., 2017; Franck et al., 2024; Bois et al., 2024) and *correction* methods that augment solver outputs (Kochkov et al., 2021; Um et al., 2020; Pathak et al., 2020; MacArt et al., 2021). We focus on correction methods, particularly under incomplete physics where the PDE is only partially known and the numerical solver captures only a subset of the underlying physical operators.

Despite their promise, current hybrid models face several

limitations: (1) training instability when back-propagating through heterogeneous solver–network architectures (List et al., 2025), (2) limited treatment of incomplete physics where parts of the PDE are missing or uncertain, and (3) empirical architecture designs lacking principled frameworks. While differentiable solvers (Um et al., 2020; Kochkov et al., 2021) enable gradient backpropagation through PDE operators, differentiability alone does not resolve how to optimally combine known physics with learned corrections. We address these challenges by repurposing proximal splitting algorithms as hybrid neural architectures that combine *plug-and-play* numerical solvers with neural components trained end-to-end (Um et al., 2020; Wei et al., 2025), rather than as standalone or pre-trained modules. From a methodological standpoint, this raises the question of how to leverage plug-and-play approaches for dynamics learning.

Plug-and-Play methods (PnP) (Venkatakrishnan et al., 2013; Meinhardt et al., 2017; Chan et al., 2016; Ryu et al., 2019; Kamilov et al., 2023) have become a prominent paradigm for solving ill-posed inverse problems in image processing. Initially proposed to replace proximal operators with fixed or pre-trained Gaussian denoisers, these approaches have since evolved into a broad family of algorithms. Notable variants include *Regularization by Denoising* (RED) (Romano et al., 2017) as well as Bregman-PnP approaches (Al-Shabili et al., 2022; Hurault et al., 2023) which extend classical PnP schemes beyond the Euclidean setting. However, the use of PnP principles for the direct time integration of evolution PDEs, or for the design of hybrid PDE solvers, has received little to none attention. To the best of our knowledge, ours is the first work to investigate this connection.

Contributions. This paper introduces a proximal optimization framework for hybrid scientific machine learning, which reformulates the coupling of known physics and learned components as a structured splitting of incomplete or misspecified energy functionals. Building on this perspective, we derive two complementary families of hybrid iterative architectures based on Douglas-Rachford and forward-backward splitting, in which numerical PDE solvers are integrated in a plug-and-play manner while neural correction operators are trained jointly inside the iterative loop. We show that these schemes naturally encompass and generalize existing solver-in-the-loop and correction-based approaches, and we clarify the roles of pre- and post-correction mecha-

¹Université Jean Monnet Saint-Etienne, CNRS, Institut d’Optique Graduate School, Inria, Laboratoire Hubert Curien UMR 5516, F-42023, SAINT-ETIENNE, France ²Institut Universitaire de France (IUF). Correspondence to: Abdel-Rahim Mezidi <abdel.rahim.mezidi@univ-st-etienne.fr>.

nisms through a unified optimization viewpoint. We further propose two training strategies connecting proximal iterations to physical time integration, enabling both grounded one-step learning and trajectory-level supervision. Finally, we demonstrate the effectiveness of the proposed framework on a range of PDE benchmarks, showing improved accuracy and stability over standard hybrid baselines, particularly in regimes characterized by incomplete or uncertain physics.

Outline. The rest of the paper is organized as follows. We present the motivation underpinning our work in Section 2, introduce the hybrid proximal models in Section 3, describe training strategies as well as variants in Section 4, and present experiments in Section 5.

2. Motivation

In this section, we recall key concepts from PDE theory and optimization that serve as the building blocks of our proposed framework.

2.1. From PDE resolution ...

For simplicity, we restrict our attention to partial differential equations that can be written as (sub-)gradient flows of an energy functional with respect to a suitable metric. Nonetheless, similar findings can be obtained for more general PDEs discussed in the supplementary material.

Let $\mathcal{E}: \mathcal{U} \rightarrow \mathbb{R}$ be an energy functional defined on a suitable function space $\mathcal{U}$, typically of the form $\mathcal{E}(u) = \int_{\Omega} e(u(x), \nabla u(x)) \, dx$, where e denotes a local energy density. When $\mathcal{E}$ is sufficiently smooth, the evolution of the state $u: \mathbb{R}_+ \times \Omega \rightarrow \mathbb{R}^m$ takes the form of a gradient flow

$$(\forall x \in \Omega, \forall t \in \mathbb{R}_+), \quad \partial_t u(x, t) = -\frac{\delta \mathcal{E}}{\delta u}(u)(x), \quad (1)$$

where $\frac{\delta \mathcal{E}}{\delta u}$ denotes the variational derivative of $\mathcal{E}$ with respect to u . More generally, when $\mathcal{E}$ is convex but possibly non-smooth, the evolution can be formulated as a subgradient flow $\partial_t u(t) \in -\partial \mathcal{E}(u(t))$, where $\partial \mathcal{E}$ denotes the subdifferential of $\mathcal{E}$. In the smooth case, the subdifferential reduces to the variational derivative, and both formulations coincide. In both settings, the system evolves according to a steepest-descent dynamics of the energy in an infinite-dimensional function space.

Example 2.1 (Allen-Cahn). The Allen-Cahn equation, which will be intensively studied in Section 5.1, is a prototypical example of a gradient flow associated with a smooth but non-convex energy functional. It reads $\partial_t u = \Delta u - W'(u)$, where $W: \mathbb{R} \rightarrow \mathbb{R}$ is a double-well potential, typically of the form $W(u) = \frac{1}{4}(u^2 - 1)^2$. This evolution corresponds to the L^2 -gradient flow of the energy

$$\mathcal{E}(u) = \int_{\Omega} \left(\frac{1}{2} |\nabla u(x)|^2 + W(u(x)) \right) \, dx. \quad (2)$$

Although $\mathcal{E}$ is not convex due to the presence of the double-well potential, it is sufficiently smooth for the variational derivative to be well-defined.

Hereafter, we consider the (smooth) gradient flow associated with $\mathcal{E}$ and endow the space $\mathcal{U}$ with the $L^2(\Omega)$ inner product, so that the gradient is taken with respect to the L^2 metric. This choice is made for clarity and simplicity since other metrics lead to analogous formulations with different metric-induced distances.

A standard way to discretize the resulting gradient-flow dynamics in time is via an implicit Euler scheme. Given a time step $\tau > 0$ and a current iterate u_t , this yields $\frac{u_{t+1} - u_t}{\tau} = -\frac{\delta \mathcal{E}}{\delta u}(u_{t+1})$, which can be rewritten as the non-linear resolvent equation $u_{t+1} = (\text{Id} + \tau \frac{\delta \mathcal{E}}{\delta u})^{-1}(u_t)$. This implicit update is equivalently characterized by the variational problem

$$u_{t+1} \in \underset{v \in \mathcal{U}}{\operatorname{argmin}} \tau \mathcal{E}(v) + \frac{1}{2} \|v - u_t\|_{L^2(\Omega)}^2. \quad (3)$$

In practice, u is represented through a finite-dimensional discretization, so that the variational problem is solved in $\mathbb{R}^m$ with the $L^2(\Omega)$ norm reducing to a Euclidean norm.

This variational time discretization is classical in the PDE literature under the name of *minimizing movements*, and corresponds, in optimization (Parikh & Boyd, 2014; Bauschke et al., 2011), to evaluating the *proximal operator* of $\tau \mathcal{E}$, that is, $u_{t+1} = \operatorname{prox}_{\tau \mathcal{E}}(u_t)$.

2.2. ... To Proximal Splitting, Plug-and-Play and Learnable Components Inside-the-Loop

In many instances, the proximal operator $\operatorname{prox}_{\tau \mathcal{E}}$ does not admit a closed-form expression and must be approximated by solving the variational problem in (3). Alternatively, when the energy admits a decomposition into multiple components, e.g., $\mathcal{E} = \mathcal{E}_1 + \mathcal{E}_2$, with different analytical or computational properties, a common strategy consists in resorting to *proximal splitting* methods (Combettes & Pesquet, 2011). These approaches exploit the structure of the decomposition by applying the proximal operator of one component, while treating the other either explicitly (via its gradient) or implicitly (via its proximal operator), depending on its properties. Well-known instances include Douglas-Rachford and forward-backward schemes, which will be discussed in Sections 3.1 and 3.2, respectively.

A further relaxation of proximal splitting consists in replacing exact proximal operators by generic mappings that are not required to correspond to a true proximal map. This viewpoint underlies *plug-and-play* methods (Venkatakrishnan et al., 2013), in which a proximal step is substituted by a prescribed operator (most commonly a handcrafted or pre-trained denoiser) while retaining the surrounding algorithmic structure.

In the classical PnP setting, these operators are designed to act as implicit priors and are largely agnostic to any underlying physical dynamics. In contrast, we consider a different regime in which the substituted operators are *numerical solvers* that approximate known parts of an evolution equation. This shift preserves a direct connection to energy-based or variational modeling, while enabling flexible hybrid designs in which physically grounded solver steps are interleaved with learned components.

Beyond plug-and-play schemes relying on fixed or pre-trained operators, a complementary line of work, often referred to as *inside-the-loop* or *solver-in-the-loop* learning in the context of hybrid differentiable physical solvers (Um et al., 2020; Wei et al., 2025), aims at learning parts of the iterative process itself through end-to-end training. In this setting, one or several components of the update rule are parameterized by neural networks and optimized jointly while interacting with numerical solvers. This perspective naturally accommodates hybrid designs, in which structure-preserving numerical solvers are combined with learned operators, allowing one to retain physical or variational structure where available while learning only the components that are difficult to model explicitly.

In the next section, we propose a hybrid iterative framework rooted in proximal splitting, which generalizes plug-and-play ideas and allows for learning selected components of the update rule inside the loop. We argue that this framework provides principled foundations for scenarios in which the energy admits a decomposition $\mathcal{E} = \mathcal{E}_1 + \mathcal{E}_2$, including:

- (*Partial physical knowledge.*) $\mathcal{E}_1$ encodes known physical principles or governing equations, while $\mathcal{E}_2$ represents an unknown component to be learned from data.
- (*Real data fitting.*) $\mathcal{E}_1$ corresponds to an idealized physical model, whereas $\mathcal{E}_2$ captures systematic discrepancies between the model and real-world observations.
- (*Coarse numerical modeling.*) The proximal operator associated with $\mathcal{E}_1$ is only available through a coarse or approximate numerical solver, and $\mathcal{E}_2$ is learned to compensate for the resulting approximation errors.

We now describe this framework in detail by leveraging two well-known splitting algorithms.

3. Hybrid Proximal Splitting Framework

We now consider an optimization problem of the form,

$$u_{t+1} = \operatorname{argmin}_{v \in \mathbb{R}^m} \underbrace{\frac{1}{2} \|v - u_t\|^2 + \tau \mathcal{E}_1(v)}_{f(v)} + \underbrace{\tau \mathcal{E}_2(v)}_{g(v)} \quad (4)$$

where $\mathcal{E}_1$ denotes a known energy term, possibly depending on the current iterate u_t , and $\mathcal{E}_2$ represents a component

to be learned from data. For clarity, we restrict our attention to quadratic distance terms $\frac{1}{2} \|v - u_t\|^2$. This choice corresponds to the $L^2(\Omega)$ metric at the function level and yields the standard ℓ_2 norm after discretization. Alternative metrics lead to analogous formulations involving different metric-induced distance terms, and can be handled within the same framework.

In the following, we consider two popular operator splitting schemes based on the decomposition of the objective in (4) as $f + g$. This choice is motivated by the separation of the known component f from the unknown component g , which needs to be learned from data.

3.1. Hybrid Douglas-Rachford Model

Proximal splitting. The Douglas-Rachford algorithm is particularly well suited to optimization problems involving the sum of two possibly non-smooth functions f and g . Originally introduced in (Douglas & Rachford, 1956) to numerically solve heat conduction problems, it has since been extensively studied and generalized, with strong convergence guarantees established under broad conditions (Eckstein & Bertsekas, 1992; Combettes & Pesquet, 2008; Borwein & Sims, 2011).

In the context of proximal splitting, Douglas-Rachford provides a flexible framework in which both terms of the objective are handled implicitly through their proximal operators. In its simplest form, for a relaxation parameter $\gamma \in (0, 2)$, step-size $\lambda > 0$, the iterates read for all $k \in \mathbb{N}_+$:

$$\begin{aligned} v^{(k)} &= \operatorname{prox}_{\lambda f}(w^{(k)}), \\ w^{(k+1)} &= w^{(k)} + \gamma \left(\operatorname{prox}_{\lambda g}(2v^{(k)} - w^{(k)}) - v^{(k)} \right). \end{aligned} \quad (5)$$

In the convex setting, Douglas-Rachford splitting enjoys strong global convergence guarantees of $v^{(k)} \rightarrow u_{t+1}$. In non-convex scenarios, convergence can still be established under additional regularity assumptions, typically towards critical points, although global guarantees are generally lost and partial global convergence results have only recently emerged (Li & Pong, 2016; Themelis & Patrinos, 2020).

In this work, we depart from a purely optimization-theoretic perspective and leverage the Douglas-Rachford updates in (5) as a building block for designing hybrid iterative architectures that combine plug-and-play numerical solvers with learnable components trained inside the loop.

Plug-and-play. Following the discussion done in Section 2.1, for gradient-flow PDEs, performing one-step of the implicit Euler scheme can be seen as applying a proximal operator. Here, we allow for greater flexibility by plugging any type of numerical solver in place of $\operatorname{prox}_{\mathcal{E}_1}$.

However, $\mathcal{E}_1$ only appears indirectly in (5) through $\operatorname{prox}_{\lambda f}$ where $f = \frac{1}{2} \|\cdot - u_t\|^2 + \tau \mathcal{E}_1$. By using standard proximal

identities (Combettes & Pesquet, 2011), we have that

$$\text{prox}_{\lambda f} = \text{prox}_{\frac{\lambda\tau}{1+\lambda}\mathcal{E}_1} \left(\frac{\cdot + \lambda u_t}{1 + \lambda} \right), \quad (6)$$

$$\simeq \text{NS}_{\frac{\lambda\tau}{1+\lambda}} \left(\frac{\cdot + \lambda u_t}{1 + \lambda} \right), \quad (7)$$

where NS_η denotes a numerical solver replacing the proximal operator of $\eta\mathcal{E}_1$. Note that this substitution amounts to using a solver with an effective time step $\lambda\tau/(1 + \lambda)$.

Learnable component. We now draw our attention to the unknown part, namely $\mathcal{E}_2$ within g . Instead of parametrizing $\mathcal{E}_2$ directly, we propose to directly parametrize $\text{prox}_g = \text{prox}_{\tau\mathcal{E}_2}$ by a neural network $\text{NN}_\theta: \mathbb{R}^m \rightarrow \mathbb{R}^m$, with parameters $\theta \in \Theta$, playing the role of a mapping.

This replacement has several consequences, since NN_θ is not required to satisfy the properties associated with exact proximal operators (a contrario, see works of (Fang et al., 2024)). In the convex case, this includes firm non-expansiveness and the interpretation as the resolvent of a maximal monotone operator. In the non-convex case, this includes local well-posedness, stability, or the existence of an underlying energy whose proximal map is recovered. As a result, the resulting update no longer corresponds to the exact minimization of a well-defined objective, but instead defines a learned iterative mapping inspired by proximal splitting.

Despite this departure from exact proximal structure, the resulting parametrization offers substantial expressivity, making it applicable to a broad class of problems where the underlying energy is unknown, misspecified, or difficult to model explicitly.

Model HYBDR. The resulting model is obtained by running the Douglas–Rachford iteration for a fixed number K of steps and interpreting the outcome as a single update between successive iterates: $u_{t+1} \approx \text{HYBDR}(u_t)$.

Given an input u_t , the output of the model is defined as $\text{HYBDR}(u_t) := v^{(K)}$ where the sequence $\{(w^{(k)}, v^{(k)})\}_{k=1}^K$ is generated according to

$$\begin{aligned} w^{(0)} &= u_t, \\ \text{For } k \in \{0, \dots, K-1\} \\ &\left[\begin{aligned} v^{(k)} &= \text{NS}_{\frac{\lambda\tau}{1+\lambda}} \left(\frac{w^{(k)} + \lambda u_t}{1 + \lambda} \right), \\ w^{(k+1)} &= w^{(k)} + \gamma (\text{NN}_\theta(2v^{(k)} - w^{(k)}) - v^{(k)}). \end{aligned} \right. \\ v^{(K)} &= \text{NS}_{\frac{\lambda\tau}{1+\lambda}} \left(\frac{w^{(K)} + \lambda u_t}{1 + \lambda} \right), \end{aligned} \quad (8)$$

This model can be interpreted as a hybrid differentiable architecture with multiple skip-connections, rescalings, and the added input u_t incorporated at each step.

3.2. Hybrid Forward-Backward Model

Another widely used strategy consists in alternating a gradient descent step on a smooth term with a proximal step on the remaining (possibly non-smooth) term, leading to forward-backward splitting methods (Combettes, 2004; Combettes & Wajs, 2005). With the decomposition (4), two complementary variants can be considered.

Model HYBFB-POST. First, when f is differentiable with β_f -Lipschitz continuous gradient, then the following sequence converges to a minimizer of (4), under standard convexity assumptions, given that the step size $\lambda \in (0, 2/\beta_f)$.

$$\begin{aligned} v^{(k+1)} &= \text{prox}_{\lambda g} \left(v^{(k)} - \lambda \nabla f(v^{(k)}) \right) \\ &= \text{prox}_{\lambda\tau\mathcal{E}_2} \left(\lambda(u_t - v^{(k)}) + v^{(k)} - \lambda\tau\nabla\mathcal{E}_1(v^{(k)}) \right) \\ &\simeq \text{NN}_\theta \left(\lambda(u_t - v^{(k)}) + \text{NS}_{\lambda\tau}(v^{(k)}) \right) \end{aligned} \quad (9)$$

where in the last equation, we have replaced the explicit Euler scheme update $v^{(k)} - \lambda\tau\nabla\mathcal{E}_1(v^{(k)})$ with a numerical solver step, and have modeled the proximal operator by a neural network, to be trained inside the loop. The latter can be interpreted as a *post-correction*, since it appears after the application of the numerical solver.

Model HYBFB-PRE. Second, when g is differentiable with β_g -Lipschitz continuous gradient, an analogous forward-backward scheme can be considered, in which the roles of f and g are exchanged. In this case, the iterates are given by

$$\begin{aligned} v^{(k+1)} &= \text{prox}_{\lambda f} \left(v^{(k)} - \lambda \nabla g(v^{(k)}) \right) \\ &= \text{prox}_{\frac{\lambda\tau}{1+\lambda}\mathcal{E}_1} \left(\frac{v^{(k)} + \lambda u_t - \lambda\tau\nabla\mathcal{E}_2(v^{(k)})}{1 + \lambda} \right) \\ &\simeq \text{NS}_{\frac{\lambda\tau}{1+\lambda}} \left(\frac{v^{(k)} + \lambda u_t}{1 + \lambda} + \text{NN}_\theta(v^{(k)}) \right), \end{aligned} \quad (10)$$

where, in the last line, the proximal operator associated with $\mathcal{E}_1$ is replaced by a numerical solver, while the gradient $\nabla\mathcal{E}_2$ is explicitly modeled by a neural network (up to a scaling factor). A contrario, here the neural network serves the role of a *pre-correction* being applied before being fed to the numerical solver.

Remark 3.1 (Convex vs. non-convex case). For clarity, the above convergence statements and step-size conditions are stated under standard convexity assumptions. However, forward-backward and related splitting schemes have also been studied in non-convex settings (Bolte et al., 2014; Boj et al., 2016), where convergence can still be established under additional regularity assumptions, typically towards critical points rather than global minimizers. In such cases, step-size conditions and stability guarantees depend more

delicately on local smoothness and regularity properties of the underlying operators, and global convergence guarantees are generally lost. In the presence of learned components, such non-convexity is unavoidable, further motivating algorithmic designs that remain stable under weaker assumptions.

An important distinction between the two forward-backward variants lies in the conditions governing the step size λ . In the first case, the step size is constrained by the stability and regularity properties of the numerical solver associated with $\mathcal{E}_1$, which is fixed and can be analyzed independently of the learning process. In contrast, in the second case, admissible step sizes depend on the regularity of the learned network modeling $\nabla \mathcal{E}_2$.

4. Training Strategies and Variants

We now turn to the learning aspects of the framework. In particular, while the models define parametric update rules, we have not yet specified how their parameters are trained from data. This section addresses this question and discusses two complementary learning strategies. The first strategy corresponds to the primary setting considered in this work, in which the model is trained to approximate a single update of the form $u_t \mapsto u_{t+1}$. The second strategy, closer in spirit to several existing approaches, considers a time-unrolled variant in which the internal iterations of the scheme are explicitly aligned with physical time steps and trained accordingly.

4.1. Single-Step Training

We first consider the more *grounded* setting, in which any of the proposed hybrid models is trained to map a current state u_t to the next state u_{t+1} .

We denote by HYBPROX any of such model, and let $\bar{\theta} \in \bar{\Theta}$ the augmented set of learnable parameters. This set naturally includes the weights θ of the neural network NN_θ but can also capture algorithmic parameters such as $\gamma \in (0, 2)$ in (8) or $\lambda > 0$ in (8) to (10).

Given a dataset of n state-transition pairs $\{u_-^{(i)}, u_+^{(i)}\}_{i=1}^n$, the parameters are learned to minimize the ℓ_2 reconstruction errors, namely

$$\underset{\bar{\theta} \in \bar{\Theta}}{\text{minimize}} \sum_{i=1}^n \left\| u_+^{(i)} - \text{HYBPROX}(u_-^{(i)}; \bar{\theta}) \right\|^2. \quad (11)$$

In this setting, only the final output of the hybrid scheme is supervised, while intermediate iterates are treated as latent computational states.

In practice, direct access to paired state transitions may not always be available. In such cases, the learned operator

must be applied repeatedly in an auto-regressive fashion in order to generate predictions over longer horizons, i.e., $u_{t+n} \approx \text{HybProx}^{\circ n}(u_t)$. This induces a deep computational graph composed of repeated applications of the same hybrid operator, and training or deployment in this regime raises additional challenges related to stability, error accumulation, and computational cost. These considerations motivate alternative training strategies in which the internal iterations of the model are explicitly unrolled and aligned with intermediate time steps, as discussed next.

4.2. Time-Unrolled Training

We now consider a complementary learning strategy in which the proposed hybrid schemes are trained to predict the system evolution over multiple time steps. In contrast to the single-step setting, the model is here designed to map, for each trajectory, the initial state u_0 to the sequence of states $u_1, u_2, \dots, u_T$, for some fixed horizon $T \geq 1$.

To this end, we set the number of internal iterations equal to the prediction horizon, i.e., $K = T$, and interpret the internal iterations of the proximal splitting algorithms as successive time steps. This produces intermediate states corresponding to $u_1, u_2, \dots, u_T$. However, directly associating all internal variables of the algorithm with physical states is generally not appropriate and requires some care. In our setting, we consider as *physical* only the outputs of the numerical solver, which approximates the underlying PDE dynamics. Accordingly, whenever a numerical solver is applied with an effective time increment η , the physical time is advanced by η .

Remark 4.1 (Choice of step-size). To ensure consistency between the number of internal iterations and the targeted physical time horizon, we select the step size λ such that each call to the numerical solver advances the system by a time increment τ/T . Specifically, we use

$$\lambda = \begin{cases} 1/(T-1), & \text{for HYBDR and HYBFB-PRE,} \\ 1/T, & \text{for HYBFB-POST.} \end{cases} \quad (12)$$

With these choices, the numerical solver is called with an effective time step equal to $\lambda\tau/(1+\lambda) = \tau/T$ and $\lambda\tau = \tau/T$, respectively. As a result, the T internal iterations correspond to T equally spaced time intervals of size τ/T , collectively approximating the evolution from u_0 to u_T .

We are now ready to describe the optimization strategy in the time-unrolled setting. Given a dataset of n trajectories $\{(u_0^{(i)}, u_1^{(i)}, \dots, u_T^{(i)})\}_{i=1}^n$, the model parameters are learned by minimizing a multi-step reconstruction loss of the form

$$\underset{\bar{\theta} \in \bar{\Theta}}{\text{minimize}} \sum_{i=1}^n \sum_{t=1}^T \left\| u_t^{(i)} - v_t^{(i)}(\bar{\theta}) \right\|^2. \quad (13)$$

where $\{v_i^{(t)}(\bar{\theta})\}_{t=1}^T$ denote the T successive iterates produced by the unrolled application of HYBPROX initialized at $u_0^{(i)}$.

4.3. Discussion

Role of the step size λ . From a pure optimization perspective, as discussed in Remark 3.1, λ should be selected to guarantee convergence of the underlying algorithmic iterates to a fixed point. In that setting, HYBDR is the most robust option, since its convergence is guaranteed for any $\lambda > 0$ under standard assumptions. Among the forward-backward variants, HYBFB-POST is the next most favorable choice, in line with the discussion following Remark 3.1.

In contrast, here the iterative scheme is no longer run until convergence but is truncated after a fixed number of iterations and trained end-to-end. As a result, λ should primarily be understood as a modeling and discretization parameter that can also control the effective time advancement and numerical behavior of each iteration, rather than as a step size chosen solely for asymptotic convergence guarantees. This shift in perspective allows greater flexibility in the choice of λ , while still benefiting from the structural stability properties of the underlying splitting scheme.

Single-step vs. time-unrolled. The choice between single-step and time-unrolled training reflects a trade-off between computational efficiency and trajectory-level supervision. Single-step training yields shallow computational graphs and simpler optimization, and is well suited to smooth dynamics approaching a near steady state. In contrast, time-unrolled training incurs higher memory and computational costs but enables learning more complex or unstable dynamics with explicit supervision over long-term horizons.

Connections with existing hybrid models. Interestingly, the proposed time-unrolled hybrid forward-backward schemes exhibit close connections with *Solver-In-The-Loop* (SITL) models. In particular, HYBFB-POST and HYBFB-PRE can be seen as structured generalizations of SITL (Um et al., 2020) and its pre-correction variant SITL* (Wei et al., 2025), respectively. More precisely, defining the affine correction $\delta^{(t)} := (u_0 - v^{(t)})/T$, we obtain

$$\begin{aligned} \text{(HYBFB-POST)} \quad v^{(t+1)} &= \text{NN}\left(\text{NS}(v^{(t)}) + \delta^{(t)}\right), \\ \text{(SITL)} \quad v^{(t+1)} &= \text{NN}\left(\text{NS}(v^{(t)})\right), \end{aligned}$$

and

$$\begin{aligned} \text{(HYBFB-PRE)} \quad v^{(t+1)} &= \text{NS}\left(v^{(t)} + \delta^{(t)} + \text{NN}(v^{(t)})\right), \\ \text{(SITL*)} \quad v^{(t+1)} &= \text{NS}\left(\text{NN}(v^{(t)})\right). \end{aligned}$$

where all subscripts are omitted for clarity, notably since all solver calls use the same time increment. It is worth noting that the proposed HYBFB models only differ by additional skip connections involving both the input u_0 and the previous iterate $v^{(t)}$, while sharing the same solver and network components. As a result, performance differences can be attributed to architectural coupling choices rather than modeling capacity.

5. Experiments

We propose to evaluate the effectiveness of our framework for PDE according to two different solver correction settings. In the first one, we consider a partial physical knowledge setting in the case where the PDE can be expressed as a direct gradient flow of an energy functional. In the second one, we study a case where the PDE does not admit this expression, and we study the interest of our approach where the solver is only able to provide a coarse approximation of the PDE that needs to be corrected from data.

5.1. Solving PDE with Partial Physical Knowledge

For this setting, we consider the 2D Allen-Cahn equation ($\partial_t u = \epsilon^2 \nabla^2 u + u - u^3$). As evoked in Section 2.1, this equation can be expressed as a gradient of flow associated with a smooth energy functional than can be easily separated into two parts (see Eq. (2)): the diffusion term $\epsilon^2 \nabla^2 u$ and the reaction one $u - u^3$ corresponding to the double-well potential. This equation is indeed particularly adapted to our setting and we propose to evaluate our method when one of the two terms is known and modeled by the solver while the other one is learned from data.

Our objective here is twofold. First, we verify that our proposed framework can recover missing physical components when only partial knowledge of the governing equation is available. Second, we assess whether our unrolled splitting architectures improves stability and accuracy compared to purely data-driven and hybrid baselines.

Following the decomposition introduced in Section 2.1, we evaluate three complementary scenarios that reflect different levels of physical knowledge. In the first scenario, DIFFUSION ONLY, the numerical solver models only the diffusion operator $\epsilon^2 \nabla^2 u$, while the neural network learns the reaction term $u - u^3$. In the second scenario, REACTION ONLY, the solver models only the reaction, and the neural network must recover the diffusion. Finally, in the BOTH TERMS setting, the solver accounts for both terms of the PDE, and the neural network acts as a corrective component that compensates for discretization errors at each unrolled step. This last configuration allows us to isolate the benefit of learned corrections even when the underlying physics is fully specified.

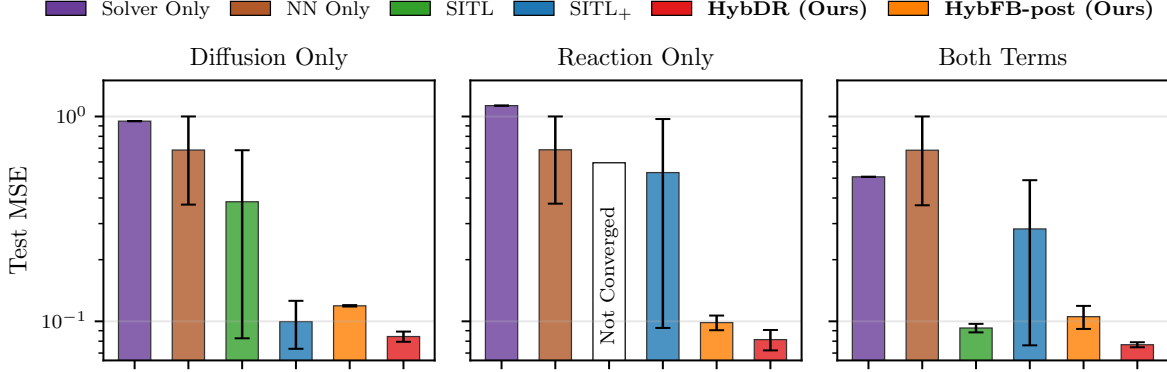


Figure 1. Mean test MSE on the Allen–Cahn equation using a Resnet architecture for different hybrid and baseline methods under three incomplete-physics settings: *Diffusion Only*, *Reaction Only*, and *Both Terms*. Error bars denote the standard error over three independent runs. In the *Reaction Only* setting, one baseline methods fail to converge, as indicated. Our hybrid proximal models (HYBDR and HYBFB-POST) consistently achieve the lowest errors across all scenarios.

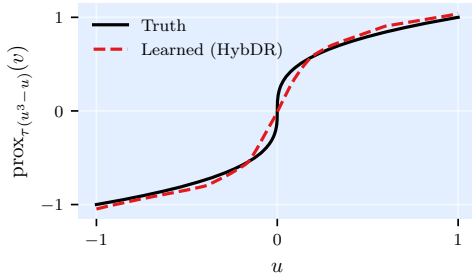


Figure 2. Learned proximal operator for the reaction term in the Allen–Cahn equation. The neural network approximation (red dashed) is compared with the exact proximal operator $\text{prox}_{\tau(u^3-u)}$ (black) for $\tau = 1$. Despite the non-convexity of the underlying energy, the learned operator closely matches the true proximal map, with a mean absolute error (MAE) of 0.053.

Dataset. We consider a 64×64 grid with $\epsilon = 0.02$. The dataset is generated by solving the full PDE with a high-precision solver starting from random initial conditions. We split the data into 700 training, 150 validation, and 150 test samples. Our task consists in learning the mapping from initial state to state after $T \times dt = 150 \times 0.1 = 15$ time units, unrolling each model $K = 10$ times (each intermediate step corresponds to 1.5 time units). Details can be found in Appendix A.1.

We compare our Forward-Backward HYBFB and Douglas-Rachford HYBDR models to several baselines. The *Solver Only* relies exclusively on the numerical solver, while the *NN Only* baseline replaces the physical model entirely with a neural network trained end-to-end. We also consider hybrid models discussed in 4.3: post-correction SITL, but also a new variant named SITL₊, corresponding to the post-correction model with an added numerical step at the end, to mimic our HYBDR architecture.

Training details. All models are trained using the Adam optimizer on MSE loss. For Allen–Cahn, we use a learning rate 10^{-3} and weight decay 10^{-6} , training for up to 200 epochs with early stopping (patience 20 epochs) on validation loss. We consider ResNets models with 64 hidden channels. Each experiment is repeated across four fixed seeds.

Results. Figure 1 summarizes the performance across all methods and scenarios. Our proposed HYBDR model achieves the lowest MSE across all three scenarios.

In the *DIFFUSION ONLY* scenario, HybDR achieves MSE $5.70 \times 10^{-3} \pm 6.26 \times 10^{-4}$, significantly outperforming SITL₊ ($9.03 \times 10^{-3} \pm 4.72 \times 10^{-3}$) and SITL ($2.62 \times 10^{-1} \pm 2.57 \times 10^{-1}$).

In the *REACTION ONLY* scenario, several baseline methods (SITL and SITL₊) failed to converge, while HybDR maintains stable performance with MSE $5.42 \times 10^{-3} \pm 1.25 \times 10^{-3}$. This demonstrates the robustness of our splitting-based approach when critical physics components are missing.

In the *BOTH TERMS* scenario, where the solver knows the full physics, HybDR achieves MSE $4.72 \times 10^{-3} \pm 2.49 \times 10^{-4}$, outperforming all baselines. Here, the neural network corrects discretization errors after each solver step, preventing error accumulation throughout the unrolled sequence, which explains the improved performance over solver-only ($2.06 \times 10^{-1} \pm 6.32 \times 10^{-4}$).

To verify that the observed gains are independent of the neural network architecture, we repeated the Allen–Cahn experiments replacing the ResNet with a Fourier Neural Operator (FNO). Figure 3 reports the results across all three scenarios. The same qualitative pattern holds: HYBDR and HYBFB-POST consistently achieve the lowest errors. This confirms that the benefit originates from the proximal splitting structure rather than from the architecture.

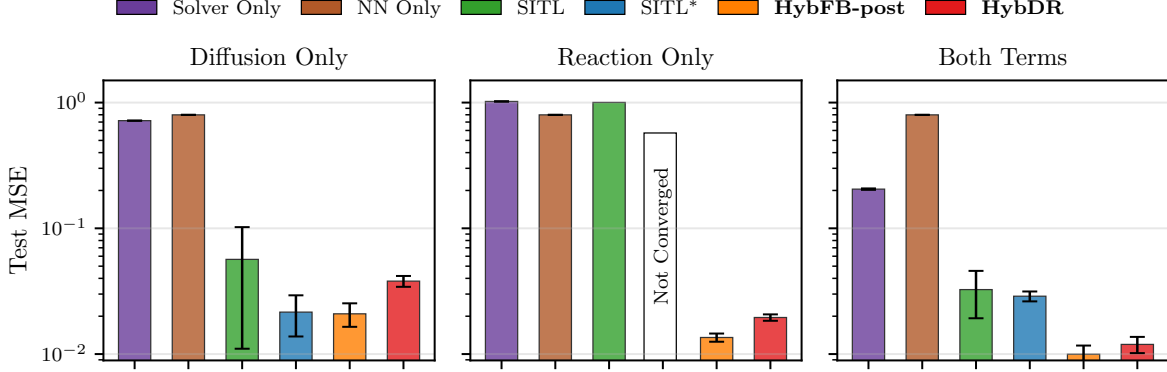


Figure 3. Mean test MSE on the Allen–Cahn equation with an FNO, under the same three settings as Figure 1. Error bars denote the standard error over three independent runs. The same qualitative behavior holds as with ResNet: HYBDR and HYBFB-POST achieve the lowest errors, and some baselines fail to converge in the *Reaction Only* setting.

Quality of the learned proximal operator. In the DIFFUSION-ONLY scenario, the neural network is trained to model the proximal operator associated with the reaction energy, namely the double-well potential. Although this energy is non-convex, its exact proximal operator can be computed and is single-valued almost everywhere, except at 0, which lies at the midpoint between the two wells. Figure 2 shows that the learned mapping closely matches the true proximal operator, indicating that the model successfully recovers the missing reaction physics.

5.2. Coarse Correction with Non-strict Gradient Flow PDE

We consider the 1D Kuramoto–Sivashinsky equation ($\partial_t u + u\partial_x u + \partial_{xx} u + \partial_{xxxx} u = 0$), is a well-known PDE for modeling chaotic dynamical systems with intrinsic instabilities. This PDE cannot be expressed as a pure gradient flow of an energy functional due to the presence of its non linear term $u\partial_x u$ which does not allow to obtain a variational form expressing the phenomenon as a unique global decreasing energy (Engelnkemper et al., 2019; Holba, 2023; Hansen et al., 2026). This property does not allow us to decompose this equation into energy terms characterizing different part of the physics as done with the Allen–Cahn equation. However, we use our setting in the coarse numerical modeling to correct the approximation of the solver from data which a very important aspect in the context of a chaotic PDE. We expect then that our hybrid approach improves numerical accuracy compared to the other hybrid model baselines.

Setup. We evaluate this equation using a pseudo-spectral solver in 1D on a 64-point grid, see Appendix A.2. The dataset consists of trajectories with $T \times dt = 100 \times 0.01 = 1$ time unit between initial and target states, unrolling each model $K = 50$ times (each intermediate step corresponds to 0.02 time units). For Kuramoto–Sivashinsky, the numerical

solver models the complete PDE, while the neural network learns discretization corrections.

Training details. We use a learning rate 10^{-4} and weight decay 10^{-6} with the same training schedule as for the Allen–Cahn equation (200 epochs, early stopping on validation with patience of 20 epochs). We also use ResNets models with 64 hidden channels and we repeat the experiments across three fixed seeds. We evaluate the MSE of each model on the test set, the intermediates outputs as training losses are also monitored to ensure stability during training.

Results. Figure 4 (left) reports the mean test MSE on the Kuramoto–Sivashinsky equation. In this more challenging regime, baseline approaches higher errors and sometimes fail to converge. In contrast, the proposed hybrid proximal models (HYBDR and HYBFB-POST) remain stable and achieve the lowest prediction errors. These results demonstrate that, although our framework is motivated by gradient-flow, its applicability extends well beyond this setting.

FNO variant. We repeated the Kuramoto–Sivashinsky experiment replacing the ResNet with an FNO. As shown in Figure 4 (right), while the proposed hybrid proximal models achieve higher test MSE than SITL, they achieve better stability across seeds. SITL* exhibits high variance.

Scaling with resolution. To assess scalability, we evaluated the proposed hybrid models on the Kuramoto–Sivashinsky equation across four spatial resolutions (128, 256, 512, and 2048 grid points). As shown in Figure 5, all three hybrid models remain stable and achieve comparable test MSE across resolutions, with only a mild increase at resolution 2048. These results demonstrate that the proposed framework scales gracefully with spatial resolution.

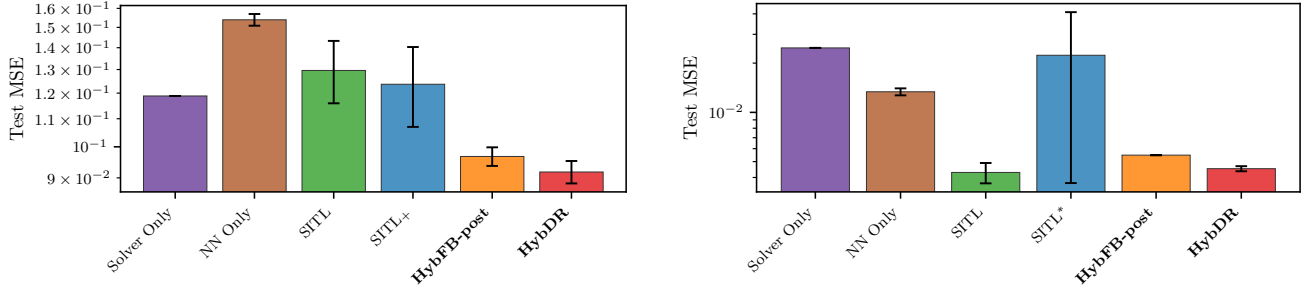


Figure 4. Mean test MSE on the Kuramoto–Sivashinsky equation for different methods, using ResNet (Left) and FNO (Right) neural networks. Error bars show standard error of the mean across three runs. HYBDR and HYBFB-POST achieve low and stable prediction errors with both architectures. Note that some SITL and SITL* runs failed to converge.

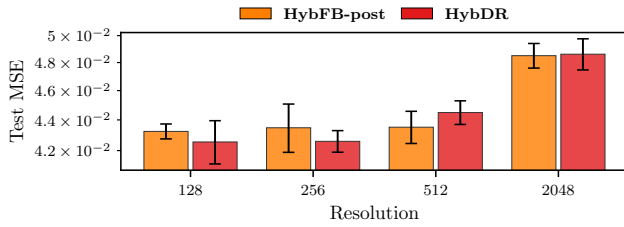


Figure 5. Test MSE of HYBFB-POST and HYBDR on Kuramoto–Sivashinsky across spatial resolutions 128 to 2048. All three methods remain stable, with only a mild degradation at resolution 2048.

6. Conclusion

We proposed a proximal optimization perspective for hybrid scientific machine learning, showing how classical splitting algorithms naturally induce hybrid architectures that combine numerical PDE solvers and learnable neural networks. By reinterpreting implicit time discretizations as proximal steps, we derived Douglas-Rachford and forward-backward hybrid models in which numerical solvers are used in a plug-and-play fashion, while the neural network is trained end-to-end. This framework unifies and generalizes several existing correction and solver-in-the-loop approaches, while providing a principled way to design new architectures and reason about pre- and post-correction mechanisms. Empirical validation on the Allen–Cahn and Kuramoto–Sivashinsky equations, demonstrates improved accuracy over standard hybrids models in the context of partial physics modeling problems or numerical accuracy improvement. This illustrates that our framework offers a new efficient hybrid modeling approach particularly suited in settings governed by incomplete or uncertain physics.

Limitations and outlook. Training may still exhibit instabilities due to unrolling depth and solver–network interactions, and the choice of step-size and relaxation parameters introduces additional hyperparameters.

A fundamental theoretical limitation is that, once iterations are truncated and operators are learned end-to-end, classical convergence guarantees no longer apply: even the existence of a fixed point is not guaranteed, and the framework should be understood as a parameterized model rather than an algorithm run to convergence. Establishing generalization bounds for such unrolled architectures remains largely open. The method also inherits the grid dependence of the numerical solver and does not provide strict resolution invariance, although Figure 5 suggests graceful degradation across resolutions.

Finally, our current experiments rely on generic neural network architectures. Future work will explore architectures more tightly aligned with the requirements necessary to define a proximal operator, either by design or by adding a regularization in the optimization objective.

Acknowledgments

This work has been funded by a public grant from the French National Research Agency (ANR) under the “France 2030” investment plan, which has the reference EUR MANUTECH SLEIGHT - ANR-17-EURE-0026.

This research was funded in whole or in part by the French National Research Agency (ANR) under projects number ANR-24-CE23-7140-01 and ANR-25-CE23-5511-01.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Al-Shabili, A. H., Xu, X., Selesnick, I., and Kamilov, U. S. Bregman plug-and-play priors. In *2022 IEEE International Conference on Image Processing (ICIP)*, pp. 241–245. IEEE, 2022.
- Bauschke, H. H., Burachik, R. S., Combettes, P. L., Elser, V., Luke, D. R., and Wolkowicz, H. *Fixed-point algorithms for inverse problems in science and engineering*, volume 49. Springer Science & Business Media, 2011.
- Bois, L., Franck, E., Navoret, L., and Vigon, V. An optimal control deep learning method to design artificial viscosities for discontinuous galerkin schemes. *Journal of Scientific Computing*, 101(3):70, 2024.
- Bolte, J., Sabach, S., and Teboulle, M. Proximal alternating linearized minimization for nonconvex and nonsmooth problems. *Mathematical Programming*, 146(1):459–494, 2014.
- Borwein, J. M. and Sims, B. The douglas–rachford algorithm in the absence of convexity. In *Fixed-point algorithms for inverse problems in science and engineering*, pp. 93–109. Springer, 2011.
- Boţ, R. I., Csetnek, E. R., and László, S. C. An inertial forward–backward algorithm for the minimization of the sum of two nonconvex functions. *EURO Journal on Computational Optimization*, 4(1):3–25, 2016.
- Chan, S. H., Wang, X., and Elgendy, O. A. Plug-and-play admm for image restoration: Fixed-point convergence and applications. *IEEE Transactions on Computational Imaging*, 3(1):84–98, 2016.
- Combettes, P. L. Solving monotone inclusions via compositions of nonexpansive averaged operators. *Optimization*, 53(5-6):475–504, 2004.
- Combettes, P. L. and Pesquet, J.-C. A douglas–rachford splitting approach to nonsmooth convex variational signal recovery. *IEEE Journal of selected topics in signal processing*, 1(4):564–574, 2008.
- Combettes, P. L. and Pesquet, J.-C. Proximal splitting methods in signal processing. In *Fixed-point algorithms for inverse problems in science and engineering*, pp. 185–212. Springer, 2011.
- Combettes, P. L. and Wajs, V. R. Signal recovery by proximal forward-backward splitting. *Multiscale modeling & simulation*, 4(4):1168–1200, 2005.
- Douglas, J. and Rachford, H. H. On the numerical solution of heat conduction problems in two and three space variables. *Transactions of the American mathematical Society*, 82(2):421–439, 1956.
- Eckstein, J. and Bertsekas, D. P. On the douglas–rachford splitting method and the proximal point algorithm for maximal monotone operators. *Mathematical Programming*, 55(1-3):293–318, 1992. doi: 10.1007/BF01581204.
- Engelnkemper, S., Gurevich, S. V., Uecker, H., Wetzel, D., and Thiele, U. *Continuation for Thin Film Hydrodynamics and Related Scalar Problems*, pp. 459–501. Springer International Publishing, Cham, 2019.
- Fang, Z., Buchanan, S., and Sulam, J. What’s in a prior? learned proximal networks for inverse problems. In *The Twelfth International Conference on Learning Representations*, 2024.
- Franck, E., Michel-Dansac, V., and Navoret, L. Approximately well-balanced discontinuous galerkin methods using bases enriched with physics-informed neural networks. *Journal of Computational Physics*, 512:113144, 2024.
- Hansen, E., Barham, W., and Morrison, P. J. Thermodynamics of driven systems via the kuramoto-sivashinsky equation, 2026.
- Holba, P. Complete classification of local conservation laws for generalized cahn–hilliard–kuramoto–sivashinsky equation. *Studies in Applied Mathematics*, 151(1):171–182, 2023.
- Holl, P. and Thuerey, N. Φ_{Flow} : Differentiable simulations for machine learning. In *ICML 2024 Workshop on Differentiable Almost Everything: Differentiable Relaxations, Algorithms, Operators, and Simulators*, 2024.
- Hurault, S., Kamilov, U., Leclaire, A., and Papadakis, N. Convergent bregman plug-and-play image restoration for poisson inverse problems. *Advances in Neural Information Processing Systems*, 36:27251–27280, 2023.
- Kamilov, U. S., Bouman, C. A., Buzzard, G. T., and Wohlberg, B. Plug-and-play methods for integrating physical and learned models in computational imaging: Theory, algorithms, and applications. *IEEE Signal Processing Magazine*, 40(1):85–97, 2023.
- Kochkov, D., Smith, J. A., Alieva, A., Wang, Q., Brenner, M. P., and Hoyer, S. Machine learning–accelerated computational fluid dynamics. *Proceedings of the National Academy of Sciences*, 118(21):e2101784118, 2021.
- Li, G. and Pong, T. K. Douglas–rachford splitting for non-convex optimization with application to nonconvex feasibility problems. *Mathematical programming*, 159(1):371–401, 2016.

- List, B., Chen, L.-W., Bali, K., and Thuerey, N. Differentiability in unrolled training of neural physics simulators on transient dynamics. *Computer Methods in Applied Mechanics and Engineering*, 433:117441, 2025.
- MacArt, J. F., Sirignano, J., and Freund, J. B. Embedded training of neural-network subgrid-scale turbulence models. *Phys. Rev. Fluids*, 6:050502, May 2021.
- Meinhardt, T., Moller, M., Hazirbas, C., and Cremers, D. Learning proximal operators: Using denoising networks for regularizing inverse imaging problems. In *Proceedings of the IEEE International Conference on Computer Vision*, pp. 1781–1790, 2017.
- Parikh, N. and Boyd, S. Proximal Algorithms. *Found. Trends Optim.*, 1(3):127–239, January 2014. ISSN 2167-3888.
- Pathak, J., Mustafa, M., Kashinath, K., Motheau, E., Kurth, T., and Day, M. Using machine learning to augment coarse-grid computational fluid dynamics simulations. *arXiv preprint arXiv:2010.00072*, 2020.
- Romano, Y., Elad, M., and Milanfar, P. The little engine that could: Regularization by denoising (red). *SIAM journal on imaging sciences*, 10(4):1804–1844, 2017.
- Ryu, E., Liu, J., Wang, S., Chen, X., Wang, Z., and Yin, W. Plug-and-play methods provably converge with properly trained denoisers. In *International Conference on Machine Learning*, pp. 5546–5557. PMLR, 2019.
- Themelis, A. and Patrinos, P. Douglas–rachford splitting and admm for nonconvex optimization: Tight convergence results. *SIAM Journal on Optimization*, 30(1):149–181, 2020.
- Tompson, J., Schlachter, K., Sprechmann, P., and Perlin, K. Accelerating eulerian fluid simulation with convolutional networks. In *International conference on machine learning*, pp. 3424–3433. PMLR, 2017.
- Um, K., Brand, R., Fei, Y. R., Holl, P., and Thuerey, N. Solver-in-the-loop: Learning from differentiable physics to interact with iterative pde-solvers. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 6111–6122. Curran Associates, Inc., 2020.
- Venkatakrishnan, S. V., Bouman, C. A., and Wohlberg, B. Plug-and-play priors for model based reconstruction. In *2013 IEEE global conference on signal and information processing*, pp. 945–948. IEEE, 2013.
- Wei, H., Franz, A., List, B. M., and Thuerey, N. INC: An indirect neural corrector for auto-regressive hybrid PDE solvers. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- Yin, Y., Le Guen, V., Dona, J., de Bézenac, E., Ayed, I., Thome, N., and Gallinari, P. Augmenting physical models with deep networks for complex dynamics forecasting*. *Journal of Statistical Mechanics: Theory and Experiment*, 2021(12):124012, dec 2021.

A. Numerical Solver Implementations

In this section, we provide implementation details for the numerical solvers used in our experiments. These solvers serve as the incomplete physics models $\nabla E'$ (for forward-backward splitting) or $\text{prox}_{E'}$ (for Douglas-Rachford splitting). All the solvers are fully compatible with Torch auto-differentiation, allowing for the complete backpropagation of the hybrid model. This ensures that gradients can be computed through the entire hybrid architecture, allowing proper training of the neural network.

A.1. Allen-Cahn Solver

The Allen-Cahn equation is a prototypical phase-field model describing the evolution of an order parameter $u(x, t)$ in a bistable system:

$$\partial_t u = \varepsilon^2 \Delta u - (u^3 - u), \quad (14)$$

where $\varepsilon > 0$ controls the interface width and the nonlinear term $-(u^3 - u)$ represents a double-well potential favoring $u \approx \pm 1$.

Implementation. Our implementation uses PhiFlow (Holl & Thuerey, 2024), a differentiable PDE solver library, with periodic boundary conditions. The solver decomposes the Allen-Cahn dynamics into diffusion and reaction steps, allowing for flexible operator splitting:

- **Diffusion step:** We apply either explicit or implicit Euler scheme for the Laplacian term. The implicit scheme uses PhiFlow’s `diffuse.implicit` method, which solves a system of linear equations. The explicit scheme uses `diffuse.explicit`, applying a second order Euler diffusion step.
- **Reaction step:** The cubic nonlinearity is treated explicitly via forward Euler:

$$u^{n+1} = u^n + \Delta t \cdot (-(u^n)^3 + u^n). \quad (15)$$

The dataset was generated using implicit diffusion for the numerical solver with the following parameters: interface width $\varepsilon = 0.01$, time step $\Delta t = 0.02$, and grid size 64×64 . Multiple trajectories were computed from random initial conditions over 25s with checkpoints saved every 0.1s, yielding 251 temporal snapshots per trajectory.

A.2. Kuramoto-Sivashinsky Solver

The Kuramoto-Sivashinsky (KS) equation is a fourth-order nonlinear PDE exhibiting spatiotemporal chaos:

$$\partial_t u + u \partial_x u + \partial_{xx} u + \partial_{xxxx} u = 0, \quad (16)$$

where the nonlinear advection term $u \partial_x u$ couples with linear dispersion ($\partial_{xxxx} u$) and anti-diffusion ($\partial_{xx} u$) to produce complex dynamics.

Implementation. Our KS solver is adapted from the Implicit Neural Correction (INC) codebase (Wei et al., 2025) and employs a pseudo-spectral method with exponential time differencing (ETD):

Fourier discretization: Spatial derivatives are computed in Fourier space using PyTorch’s FFT routines. For a domain of size L , the wavenumbers are $k = 2\pi n/L$ with $n \in \{-N/2, \dots, N/2 - 1\}$, where N is the spatial resolution.

Linear operator: The linear part of the KS equation in Fourier space is

$$\hat{\mathcal{L}}_k = -(ik)^2 - (ik)^4, \quad (17)$$

which is integrated exactly using exponential integrators.

Nonlinear term: The advection term $u \partial_x u$ is computed in physical space, transformed to Fourier space, and includes dealiasing via the 3/2 rule to prevent aliasing errors.

Time integration: We use the first-order exponential time differencing method (ETD1). In our implementation, the linear operator is handled exactly and the φ_1 function is computed as

$$\varphi_1(L) = \frac{e^L - 1}{L}, \quad (18)$$

with the $L = 0$ case handled by $\varphi_1(0) = \Delta t$. The update from time t_n to $t_{n+1} = t_n + \Delta t$ is given by

$$\hat{u}_k^{n+1} = e^{\hat{\mathcal{L}}_k \Delta t} \hat{u}_k^n + \varphi_1(\hat{\mathcal{L}}_k \Delta t) \widehat{(u \partial_x u)}_k^n, \quad (19)$$